Behique: phases and progress review

Status as of 2026-10-30, end of day. For a read-through before more is built. Everything here is marked as measured, built but unmeasured, or not built.

34
commits on main (since 2026-10-29)

465
offline tests passing in release, onboard, crawl, ui, sandbox (re-run today)

~$40
model spend so far of about $250 credits (estimate)

5 days
to the hackathon deadline (2026-11-04, 9 PM ET, not verified)

1. What Behique is

An autonomous release-verification and visual QA tool. You give it a known-good release and a release candidate of a web app. It records what each one shows along real click paths, finds what changed, replays every change on a fresh build to prove it is real (3 of 3 on each side), finds the exact commit that caused it, and writes an evidence-backed report. **A model proposes, code verifies, a person approves.** A bug only counts when a replayable assertion proves it, and the Ship or Hold decision is always a person's.

2. Phases at a glance

Piece	What it does	Status	Evidence
Explorer (early)	Explores an unseen app, reports defects as data assertions, replays each fresh.	**Built, n=1**	28 planted bugs on a todo app; noisy results.
Release pipeline	Build a git ref by sha, record page facts, compare good vs candidate with a control, write a test, bisect.	**Built, measured**	FreshRSS 1.29.1 vs 1.30.0: 2 real changes, 3/3 + 3/3, first bad commit 1b7178a in 8 steps.
Evidence store and report	Hash-chained evidence, labelled claims, report page, person's Ship or Hold, optional AI guess with price shown first.	**Built, used**	Report page and dashboard run end to end.
Canvas senses	Pixel diff and coordinate actions for canvas apps (Music Blocks).	**Built, measured**	Fixture: 18 of 18 detectable bugs, 3/3 reproduced. Music Blocks v3.8.0 vs master run (about $0.07).
Wave 1 and 2	Several findings per run, regions, screenshots, visual runs, set-up from the dashboard.	**Built, committed**	Share and scroll scenarios unchanged by the wiring.

Phase 0: Measure	Does set-up work on real repos?	**Measured in the sandbox**	**1 of 6** proposals pass as written; 3 more work with one small fix each; 2 not diagnosed to the end.
Phase 1: Set-up v2	Richer repo facts, package-manager and env handling, repair loop, profile format v2.	**Built, tested with fakes**	Never run on a real failing repo.
Phase 2: Flow crawl	Finds click paths with no model, read-only, then compares along them.	**Built, proven on one app**	Reproduced the FreshRSS Share regression with zero hand-written steps. Tuned on FreshRSS only.
Phase 3: Seed data	Put realistic data in an app that needs it.	**Not built**	Depends on Phase 4 sidecars.
Phase 4: Isolation	Run untrusted repos in Docker with database sidecars.	**Built, measured on hardac2**	First-party app + Postgres, todomvc and umami (with Postgres) ran in it; see section 6. Uncommitted.
Phase 5: Cloning	Clone a repo from the dashboard after a person approves the exact command.	**Built, previewed only**	Refusals tested; nothing cloned through it. Your own GitHub repos only.

3. The result that matters

With no hand-written steps, the crawler plus compare reproduced the FreshRSS Share regression: the share button's `data-url` and `data-title` are now URL-encoded. The crawl visited 30 states and produced 37 steps. The two changes were confirmed 3 of 3 on the good build and 3 of 3 on the candidate. Bisect found commit `1b7178a` in 8 steps, the same commit the hand-written case gives. The same comparison started from the dashboard returned HOLD with 2 reproduced changes and a screenshot.

> **Read this as:** the mechanism works. It is **not** evidence that it works on any app, because FreshRSS is the app the crawler was tuned on. The first runs of the crawl returned `fails_both` because recorded steps shared state across flows. The fix (replay each flow alone, read-only, cookies and storage cleared between flows) is what made it honest.

4. What was built overnight (2026-10-30)

Built with Muse (Meta's model) from written specs, each file read and corrected by hand. Seven commits, all offline-tested, nothing pushed.

Commit	Contents
`894f337`	inspect-more, toolchain, repair modules (specs 13 to 15)
`a7b57bd`	Profile format v2: env, static root, build steps, installs with scripts disabled
`4de1d99`	Build runner: output tail, early-exit detection, whole process group is stopped
`b0f1cd0`	The read-only crawler and `check-app` (specs 16 to 18)
`a21da41`	Read-only, per-flow replay in confirm, facts, compare, findings, shots
`13dcaa4`	Clone flow and crawl runs in the dashboard server (spec 19)
`ea6e14a`	Set-up bench, baseline and overnight log, STATUS

Real bugs found in the process: Muse's code had a test design flaw, two contradictory tests, a wrong ANSI pattern and a risk filter that treated any row containing "post" as dangerous. The build runner had a genuine bug: a dev server started through yarn or npm left a child process holding its port after a stop.

5. Safety: what was blocked and why

Twice the six-repo run was denied by Claude Code's permission system as "[Code from External]": it will not let me install dependencies and start dev servers from freshly cloned third-party repos unattended, because that runs the repo's code on your Mac. I did not work around it. That is why Phase 4 was built: the same runs, executed in Docker on hardac2, were accepted, and nothing from those repos ran on the Mac.

- Behique's own safeguards: installs run with scripts disabled; commands come from an allowlist; clone and set-up run only with the fingerprint of what a person reviewed; the crawler sends only GET and aborts cross-origin requests.
- I read each proposed command list before running it. Everything still runs only when `BEHIQUE_SANDBOX` is set; without it, profile apps run as normal host processes, as before. A mistyped value is an error, never a silent fallback to the host.

6. Phase 4: untrusted repos run in Docker, with database sidecars

Decision: one Docker runner with two targets, switched by `BEHIQUE_SANDBOX`. hardac2 over ssh is the real target (x86_64, Docker 29.9.2, 24 cores, KVM available); OrbStack on the Mac is the local option (arm64, daemon was off, not used today). Full details: `docs/sandbox.md`.

How it works

- Each check gets an **internal network** (no outside access) and an open one. Sidecars (Postgres, MySQL, MariaDB, Redis, MongoDB) and the app run on the internal one; install steps also join the open one for the package registry.

- The app is reached by its container address through a tunnel, so everything else in Behique still sees `http://127.0.0.1:port`. Nothing is published to hardac2's network.
- Every container: all capabilities dropped, no-new-privileges, read-only root, non-root user (databases excepted, with a minimal capability set), memory, CPU and process limits, deleted after the check. Lifecycle scripts stay disabled on installs.
- Every docker argument is built by code from validated parts (`docker-args.mjs`); nothing from a repo, a model or a page can become a docker flag. This matters because the docker group is root-equivalent on hardac2.

Piece	Made by	Result
docker-args (policy)	Muse spec 20, then my review fixed: root users via gid, `..` paths, a missing service-user rule	**29 tests**
services (sidecars)	Muse spec 22; I fixed redis and mongo needing the same capabilities as the databases	**14 tests**
sandbox (lifecycle)	Muse spec 21; I fixed broken ANSI patterns that deleted letters, and stale-run cleanup that missed orphaned networks and volumes	**16 tests**
runner, tunnel, backend, launcher, clean	By hand	**tests with local sockets and fakes**

Measured on hardac2

Case	Result
First-party app + Postgres sidecar	**Up in 3.6 s. The app reached `db:5432` by name, ran as uid 1000, and an outbound connection to the internet failed (ENETUNREACH). Nothing left behind.**
todomvc (static)	**VERIFIED end to end (build, pictures, noise floor).**
html5-boilerplate	**Built (`npm ci` + build) and served; the "app is alive" check failed because the page is almost blank by design.**
umami v3.3.1 + Postgres	**Install, build and `next start` ran in the sandbox; `/login` answered 200. Needed 10 GB, the full Node image and a corrected start command.**
nextjs-blog, excalidraw, vitepress	**As proposed, all three fail: `-- -p {port}` breaks under yarn Berry and pnpm; excalidraw's workspace command does not fit that version; vitepress not diagnosed.**

Bugs the real runs found (all fixed, each with a test)

- Docker copies the image folder's owner (root) onto a still-empty volume at first mount, silently undoing the permission fix. Fixed with `nocopy`.
- `/tmp` must allow execution: yarn Berry writes launchers there. Yarn Berry also refuses `--ignore-scripts`; scripts are now turned off the way each yarn version accepts.
- Dev servers that listen only on localhost were unreachable from outside the container. A small launcher inside the container relays to them.

- The default Node image lacked openssl (Prisma needs it); `next build` was killed at 4 GB (needs about 10 GB); the model's `run start -- -p {port}` breaks with pnpm and yarn Berry.
- An ssh tunnel through the shared connection exited at once; it now uses its own connection.

Limits: plain Docker (runc) is not a hard wall; before arbitrary repos, move to Firecracker or gVisor. The sidecar proposer missed umami's Postgres (its compose file names a service `db` with no hint). The repair loop is still untested on a real failure, because it needs an approval step for each changed command. Phase 3 (seed data) is not built.

7. What I would check for "tip top shape"

Run these (all safe, nothing is installed from third-party repos)

```
cd ~/dev/behique
git status                      # new: src/release/sandbox/, docs/sandbox.md, specs 20 to
22, small edits to build.mjs and profile.mjs (not committed yet)
git log --oneline | head -10    # the seven overnight commits are there
node --test src/release/*.test.mjs src/release/onboard/*.test.mjs \
  src/release/crawl/*.test.mjs src/release/ui/*.test.mjs \
  src/release/sandbox/*.test.mjs   # expect 465 pass, 0 fail
```

Look at by hand

- `docs/overnight-log.md` and `docs/onboarding-baseline.md`: the honest account of the overnight run and what is unmeasured.
- `docs/STATUS.md`: the full list of what is built and the known limits.
- `docs/specs/wave3/` (specs 13 to 22): what each Muse module was told to do, in plain English.
- The dashboard on port 8790: pick Check, then Behaviour, and look at the crawl result and the clone panel.

Known weak spots

- The crawler is tuned on one app. A second app is the real test.
- The repair loop and profile v2 are tested with fakes only.
- `~/dev/behique-ui/behique-live.html` (the dashboard page) is **not under git**; backups exist only in a scratch folder.
- A Ship or Hold decision's name is typed in, so it is attribution by honour, not identity.
- The demo script is out of date: Share now has 2 findings, plus the Music Blocks segment, the set-up flow and the crawl.

8. Open questions for you

- **Hackathon rules:** the deadline (8 or 9 PM ET), track, video length limit and whether Muse Code is expected are all unverified. Please paste them from the confirmation email or official page.
- **Order after Phase 4:** Phase 3 seed data, or update the demo script and record? With five days left I would update the demo script next, then record.
- **Credits:** Meta API about $146 left of $150; Baseten about $70 of $100 (estimates). Canvas work was allowed $10 and has used about $2. The Phase 4 Muse builds cost about $0.35 in all (three specs, one

rebuilt).

Sources: docs/STATUS.md, docs/overnight-log.md, docs/onboarding-baseline.md, git log, the hardac2 read-only check on 2026-10-30. Numbers marked estimate come from those notes and were not re-measured today, except the test count and commit count.